

MEDASSIST-AI

Milestone-1 Report

Drushhti Kuhikar

Data Collection, Exploration and Preprocessing

1. Project Description

MedAssist-AI: Medical Symptom Analysis and Disease Prediction System

MedAssist-AI is an Artificial Intelligence (AI) powered healthcare platform designed to assist users in analyzing their symptoms, predicting possible diseases, assessing health risks, and providing preliminary healthcare recommendations. The primary objective of the system is to help users understand potential health conditions at an early stage and support informed healthcare decisions through AI-driven medical insights.

The platform is designed to support various healthcare functionalities, including symptom analysis, disease prediction, risk assessment, treatment recommendations, health report generation, and patient history management. It can be utilized by patients, healthcare providers, clinics, hospitals, and telemedicine platforms as an intelligent decision-support system.

The project integrates multiple healthcare datasets, such as the Disease Symptoms and Patient Profile Dataset, Disease Prediction Using Symptoms Dataset, and the CDC Behavioral Risk Factor Surveillance System (BRFSS) dataset, to build a comprehensive disease prediction model. These datasets provide information related to symptoms, diseases, patient profiles, lifestyle indicators, and healthcare conditions, enabling the system to make reliable predictions.

The complete MedAssist-AI platform consists of several modules, including User Management, Symptom Analysis, Disease Prediction, Risk Assessment, Treatment Recommendation, Health Reports, and Analytics Dashboard. Together, these modules provide an end-to-end intelligent healthcare solution.

2. Objectives

The objectives of this milestone are:

- To collect healthcare datasets from multiple sources.
- To understand the structure and characteristics of each dataset.
- To perform exploratory data analysis (EDA).

- To identify and remove duplicate records.
- To analyze missing values in the datasets.
- To clean and preprocess the collected datasets.
- To transform symptom information into machine learning compatible binary features.
- To compare symptom and disease information across different datasets.
- To identify common diseases available in multiple datasets.
- To merge compatible datasets into a single preprocessed dataset.
- To prepare a clean dataset for disease prediction model development.

3. Datasets Used

The MedAssist-AI project utilizes multiple healthcare datasets because no single dataset provides all the information required for disease prediction. Each dataset contributes different healthcare information such as symptoms, diseases, patient profiles, health conditions, and risk-related data.

3.1 CDC Behavioral Risk Factor Surveillance System (BRFSS) Dataset

The CDC BRFSS dataset is a large-scale public healthcare survey dataset containing health-related information collected from thousands of individuals. It includes demographic information, lifestyle indicators, and various medical conditions.

This dataset was primarily explored for diabetes-related health information and healthcare analytics. It contains approximately **441,456 records and 330 features**, making it the largest dataset used during the project.

Purpose

- Healthcare data exploration
- Diabetes-related analysis
- Risk assessment
- Healthcare analytics

3.2 Disease Prediction Using Symptoms Dataset

This dataset contains diseases together with their associated symptoms. Symptoms are stored across multiple columns named **Symptom_1** to **Symptom_17**, where each row represents a disease and its corresponding symptoms.

Since machine learning algorithms require numerical input rather than textual symptom values, this dataset was transformed into binary symptom features during preprocessing.

Purpose

- Symptom analysis
- Disease prediction
- Feature engineering

3.3 Disease Symptoms and Patient Profile Dataset

This dataset contains disease names together with patient profile information. It was mainly used to compare disease names with the Disease Prediction dataset and identify common diseases between different data sources.

Purpose

- Disease validation
- Patient profile analysis
- Dataset integration

3.4 Symptom Description Dataset

This dataset provides textual descriptions for different diseases. Although it was not directly used during preprocessing, it will be integrated into the final system to display disease-related information after prediction.

Purpose

- Disease description
- User guidance

3.5 Symptom Precaution Dataset

This dataset contains precautionary measures corresponding to different diseases.

Purpose

- Healthcare recommendations
- Preventive guidance

3.6 Symptom Severity Dataset

The Symptom Severity dataset assigns severity scores to different symptoms. These severity values can later be used for risk assessment and prioritizing disease predictions.

Purpose

- Severity analysis
- Risk assessment
- Future enhancement

4. Project Folder Structure

To maintain an organized workflow and ensure efficient data management, the project was structured into separate folders based on their functionality. This organization simplifies data handling, preprocessing, and model development while improving code readability and maintainability.

The project consists of the following folders:

4.1 Datasets Folder

The **datasets** folder contains all the original healthcare datasets collected from different sources. These files remain unchanged throughout the project and serve as the primary source of data.

The datasets included in this folder are:

- **2015.csv (CDC BRFSS Dataset)** – Contains health survey data and diabetes-related information.
- **Disease Prediction Using Symptoms Dataset** – Contains diseases with symptoms listed from **Symptom_1** to **Symptom_17**.
- **Disease Symptoms and Patient Profile Dataset** – Contains patient information along with disease labels.
- **Symptom Description Dataset** – Contains textual descriptions of diseases.
- **Symptom Precaution Dataset** – Contains recommended precautions for different diseases.
- **Symptom Severity Dataset** – Contains severity scores assigned to symptoms.

Keeping the original datasets unchanged ensures that the raw data is always available for future verification and analysis.

4.2 Processed_Data Folder

The **processed_data** folder stores the datasets after cleaning and preprocessing. Instead of repeatedly performing preprocessing operations every time the notebook is executed, the processed datasets are saved separately for reuse.

The processed datasets created during Milestone-1 include:

- **diabetes_clean.csv**
- **patient_clean.csv**
- **disease_clean.csv**
- **diabetes_transformed.csv**

4.3 Notebooks Folder

The **notebooks** folder contains all Jupyter notebooks used during development.

Different notebooks were created for different stages of the project, including:

- Data Exploration (EDA)
- Data Cleaning and Preprocessing
- Dataset Transformation
- Dataset Integration
- Machine Learning Model Development

5. Data Exploration (EDA)

Before performing any preprocessing operations, Exploratory Data Analysis (EDA) was carried out to understand the structure, quality, and characteristics of each dataset. EDA helps identify issues such as missing values, duplicate records, inconsistent data formats, and differences between datasets.

The datasets were loaded into Python using the Pandas library and examined using several built-in functions.

The following functions were used during exploration:

Function	Purpose
head()	Displays the first few records of the dataset
shape	Returns the number of rows and columns

Function	Purpose
info()	Displays column names, data types, and missing values
describe()	Provides statistical summary of numerical columns
isnull().sum()	Identifies missing values in each column
duplicated().sum()	Detects duplicate records

```
print("Disease Dataset:", disease_df.shape)
print("Diabetes Dataset:", diabetes_df.shape)
print("Patient Dataset:", patient_df.shape)
print("Description Dataset:", description_df.shape)
print("Precaution Dataset:", precaution_df.shape)
print("Severity Dataset:", severity_df.shape)
```

```
Disease Dataset: (42, 133)
Diabetes Dataset: (4920, 18)
Patient Dataset: (349, 10)
Description Dataset: (41, 2)
Precaution Dataset: (41, 5)
Severity Dataset: (133, 2)
```

```
disease_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 42 entries, 0 to 41
Columns: 133 entries, itching to prognosis
dtypes: int64(132), str(1)
memory usage: 43.8 KB
```

5.1 Understanding the Datasets

Initially, each dataset was loaded into a Pandas DataFrame and inspected to understand its dimensions and overall structure.

The CDC BRFSS dataset was the largest dataset used in the project, containing **441,456 records** and **330 attributes**. Due to its large size, exploratory analysis was essential before further processing.

Similarly, the Disease Prediction dataset and the Disease Symptoms and Patient Profile dataset were examined to understand how diseases and symptoms were represented.

This analysis revealed that different datasets followed different formats. While some datasets stored symptoms as independent feature columns, others stored symptoms across multiple columns such as **Symptom_1**, **Symptom_2**, ..., **Symptom_17**. This structural difference highlighted the need for data transformation before dataset integration.

5.2 Duplicate Record Analysis

Duplicate records can negatively affect machine learning models by introducing redundancy and bias. Therefore, duplicate entries were identified using the `duplicated()` function.

The diabetes dataset originally contained **4,920 records**. After removing duplicate entries, the dataset was reduced to **304 unique records**.

Removing duplicate records ensured that each observation contributed only once during future model development.

```
for name, df in datasets.items():
    print("="*70)
    print(name)
    print("Duplicate Rows :", df.duplicated().sum())

]

=====
Disease
Duplicate Rows : 0
=====
Diabetes
Duplicate Rows : 4616
=====
Patient
Duplicate Rows : 49
=====
Description
Duplicate Rows : 0
=====
Precaution
Duplicate Rows : 0
=====
Severity
Duplicate Rows : 0
=====
```

5.3 Missing Value Analysis

After removing duplicate records, missing values were analyzed using the `isnull().sum()` function.

This step helped identify incomplete records and evaluate the quality of each dataset before further preprocessing.

Understanding missing values is important because incomplete or inconsistent data may reduce the performance of machine learning algorithms if left untreated.

```
for name, df in datasets.items():
    print("-"*70)
    print(name)

    missing = df.isnull().sum()
    print(missing[missing > 0])
    print()
```

```
=====
Disease
Series([], dtype: int64)
=====
Diabetes
Symptom_4      348
Symptom_5     1206
Symptom_6     1986
Symptom_7     2652
Symptom_8     2976
Symptom_9     3228
Symptom_10     3408
Symptom_11     3726
Symptom_12     4176
Symptom_13     4416
Symptom_14     4614
Symptom_15     4680
Symptom_16     4728
Symptom_17     4848
dtype: int64
=====
Patient
Series([], dtype: int64)
...
=====
Severity
Series([], dtype: int64)
```

5.4 Saving the Cleaned Datasets

Once the exploratory analysis and cleaning process were completed, the cleaned datasets were saved into the **processed_data** folder.

Saving intermediate datasets provides several advantages:

- Prevents repeated preprocessing.
- Reduces execution time.
- Maintains consistency across notebooks.
- Allows future stages of the project to directly use cleaned datasets.

The cleaned datasets generated during this phase include:

- **diabetes_clean.csv**
- **patient_clean.csv**
- **disease_clean.csv**

These cleaned datasets formed the foundation for the preprocessing and transformation tasks performed in the next stage.


```
diabetes_df.to_csv("../processed_data/diabetes_clean.csv", index=False)

print("✅ Diabetes dataset saved successfully!")

...  ✅ Diabetes dataset saved successfully!

patient_df.to_csv("../processed_data/patient_clean.csv", index=False)

print("✅ Patient dataset saved successfully!")

...  ✅ Patient dataset saved successfully!
```

6. Data Preprocessing

Data preprocessing is a crucial step in any machine learning project, as the quality of the input data directly affects the performance of the prediction model. Since the datasets used in the MedAssist-AI project were collected from different sources, they differed in structure, feature representation, and disease naming conventions. Therefore, several preprocessing techniques were applied to prepare a consistent and machine learning-ready dataset.

The preprocessing workflow included symptom transformation, feature comparison, disease standardization, identification of common diseases, and dataset integration.

6.1 Symptom Feature Transformation

One of the major challenges encountered during preprocessing was the representation of symptoms in the Disease Prediction Using Symptoms dataset. In this dataset, symptoms were stored across multiple columns named **Symptom_1** to **Symptom_17**, where each row represented a disease and its associated symptoms.

Although this format is suitable for human interpretation, it cannot be directly used for machine learning because algorithms require numerical input features.

To overcome this limitation, the dataset was transformed into a binary feature representation. During this process, every unique symptom was converted into an independent feature column.

For each disease record:

- A value of **1** was assigned if the symptom was present.
- A value of **0** was assigned if the symptom was absent.

After processing all records, the disease name was stored separately in the **prognosis** column, while all symptom columns became binary input features.

```
# Create an empty DataFrame with the same columns as disease_df
diabetes_transformed = pd.DataFrame(columns=disease_df.columns)

print(diabetes_transformed.shape)
```

```
... (0, 133)
```

```
# Take the first row from diabetes_df
row = diabetes_df.iloc[0]

print(row)
```

```
... Disease          Fungal infection
Symptom_1                itching
Symptom_2                skin_rash
Symptom_3      nodal_skin_eruptions
Symptom_4      dischromic _patches
Symptom_5                  None
Symptom_6                  None
Symptom_7                  None
Symptom_8                  None
Symptom_9                  None
Symptom_10                 None
Symptom_11                 None
Symptom_12                 None
Symptom_13                 None
Symptom_14                 None
Symptom_15                 None
Symptom_16                 None
Symptom_17                 None
Name: 0, dtype: str
```

```
# Empty list to store transformed rows
transformed_rows = []

# Loop through every row
for _, row in diabetes_df.iterrows():

    # Create empty row
    new_row = pd.Series(index=disease_df.columns, dtype="object")
    new_row.iloc[: -1] = 0

    # Disease name
    new_row["prognosis"] = row["Disease"].strip()

    # Symptoms
    for symptom in row[1:]:

        if pd.notna(symptom) and symptom != "None":

            symptom = symptom.strip() # ★ Remove leading/trailing spaces

            if symptom in disease_df.columns:
                new_row[symptom] = 1

    transformed_rows.append(new_row)

# Create DataFrame
diabetes_transformed = pd.DataFrame(transformed_rows)

print(diabetes_transformed.shape)
```

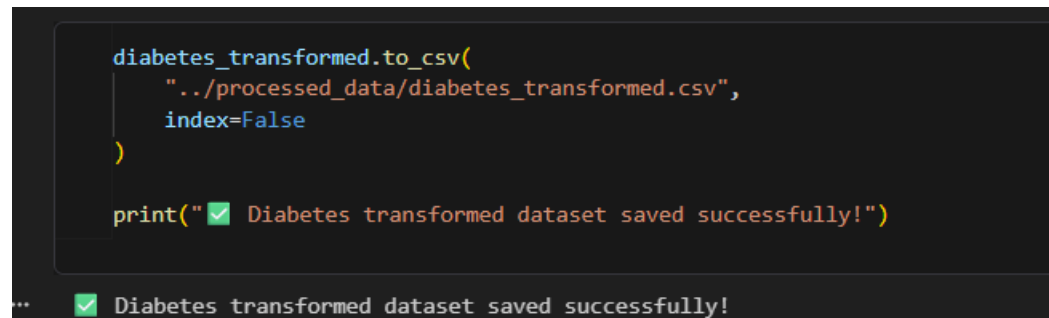
```
(304, 133)
```

6.2 Saving the Transformed Dataset

After the binary transformation was completed successfully, the transformed dataset was saved in the `processed_data` folder as:

diabetes_transformed.csv

Saving the transformed dataset separately ensures that the preprocessing stage does not need to be repeated whenever the project is executed. It also provides a reusable dataset for future machine learning tasks.



```
diabetes_transformed.to_csv(  
    "../processed_data/diabetes_transformed.csv",  
    index=False  
)  
  
print("✅ Diabetes transformed dataset saved successfully!")
```

... ✅ Diabetes transformed dataset saved successfully!

6.3 Comparison of Symptom Features

After transforming the symptom dataset, the next step was to compare the symptom features present in different datasets.

The transformed dataset contained binary symptom columns, while the Disease Prediction dataset already consisted of symptom-based feature columns. To ensure compatibility between the datasets, the names of all symptom columns were extracted and compared using Python set operations.

This comparison identified symptom features that were available in one dataset but not in the other.

The analysis revealed that the Disease Prediction dataset contained **130 additional symptom features** that were not present in the transformed diabetes dataset. This difference was expected because the Disease Prediction dataset covers a broader range of diseases, whereas the diabetes dataset focuses on a limited set of medical conditions.

Comparing symptom columns helped understand the structural differences between datasets before integration.

6.4 Disease Name Standardization

Since multiple datasets were collected from different sources, disease names were not always represented using the same format.

For example, disease names contained uppercase letters, additional spaces, or different naming conventions. Such inconsistencies can lead to incorrect comparisons during dataset integration.

To address this issue, disease names were standardized by:

- Converting all disease names to lowercase.
- Removing unnecessary spaces.
- Ensuring consistent naming across datasets.

This standardization improved the accuracy of disease matching during the integration process.

```
def clean_disease_name(name):  
    if pd.isna(name):  
        return name  
  
    name = str(name).strip().lower()  
  
    replacements = {  
        "diabetes ": "diabetes",  
        "hypertension ": "hypertension",  
        "hepatitis a": "hepatitis a",  
        "chicken pox": "chickenpox",  
        "urinary tract infection": "urinary tract infection",  
        "typhoid": "typhoid fever",  
        "heart attack": "myocardial infarction (heart attack)",  
        "bronchial asthma": "asthma",  
        "common cold": "common cold",  
        "osteoarthristis": "osteoarthritis"  
    }  
  
    return replacements.get(name, name)
```

6.5 Identification of Common Diseases

After standardizing disease names, diseases from different datasets were compared to identify common entries.

The comparison revealed that **19 diseases** were common across the datasets.

Some of the common diseases identified include:

- Diabetes
- Asthma
- Chickenpox

- Hypertension
- Pneumonia
- Tuberculosis
- Malaria
- Migraine
- Psoriasis
- Urinary Tract Infection

Identifying common diseases ensured that only compatible records were considered during dataset integration and minimized inconsistencies caused by naming differences.

```
common = set(disease_df["prognosis"]) & set(diabetes_df["prognosis"]) & set(patient_df["Disease"])

print("Common diseases:", len(common))
print(sorted(common))
```

Common diseases: 19

['acne', 'asthma', 'chickenpox', 'common cold', 'diabetes', 'gastroenteritis', 'hepatitis b', 'hypertension', ...]

6.6 Dataset Integration

Once the symptom features and disease names were standardized, the compatible datasets were combined into a single dataset.

The integration process involved:

- Matching disease names across datasets.
- Combining binary symptom features.
- Appending disease labels.
- Removing duplicate records after merging.

The Pandas library was used to concatenate the datasets into one unified dataset.

After integration, the final dataset contained:

- **306 records**
- **133 columns**

This integrated dataset represents the final output of the preprocessing stage and serves as the primary dataset for future machine learning model development.

```
# Merge both datasets

final_dataset = pd.concat(
    [disease_df, diabetes_df],
    ignore_index=True
)

print("Shape before removing duplicates:", final_dataset.shape)

[24]
... Shape before removing duplicates: (346, 133)

duplicates = final_dataset.duplicated().sum()

print("Duplicate rows:", duplicates)

[25]
... Duplicate rows: 40

# Remove duplicate rows

final_dataset = final_dataset.drop_duplicates()

print("Final Shape:", final_dataset.shape)

[26]
... Final Shape: (306, 133)

final_dataset.to_csv(
    "../processed_data/final_dataset.csv",
    index=False
)

print("✅ Final dataset saved successfully!")

[27]
... ✅ Final dataset saved successfully!
```

7. Results Achieved

The following outcomes were successfully achieved during Milestone-1:

- Multiple healthcare datasets were collected and analyzed.
- Exploratory Data Analysis (EDA) was performed to understand dataset structure and quality.
- Duplicate records were identified and removed.
- Missing value analysis was completed.
- Cleaned datasets were generated and stored.
- Symptom information was transformed from **Symptom_1–Symptom_17** into binary feature columns.

- Disease names were standardized across datasets.
- A comparison of symptom features was performed.
- **19 common diseases** were identified between datasets.
- Compatible datasets were merged successfully.
- A final preprocessed dataset consisting of **306 records and 133 columns** was prepared for future machine learning model development.

8. Conclusion

During Milestone-1, the primary objective was to prepare a clean, structured, and machine learning-ready healthcare dataset for the MedAssist-AI project. Multiple healthcare datasets were collected, explored, cleaned, transformed, and integrated through a systematic preprocessing pipeline.

A major achievement of this milestone was the successful conversion of symptom information from the **Symptom_1–Symptom_17** format into binary feature representation. This transformation enabled symptom data to be represented numerically, making it suitable for machine learning algorithms.

In addition, disease names were standardized, symptom features were compared, common diseases were identified, and compatible datasets were successfully integrated into a single dataset containing **306 records and 133 columns**.

The completed preprocessing pipeline establishes a reliable foundation for the next phase of the project, where machine learning models will be trained and evaluated for accurate disease prediction.